

DSP Processor

Architecture, FSMs, and Instruction Execution

A shared-sequencer DSP coprocessor covering FILTERING (FIR/IIR/CONV/CORR/LMS), SAMPLING (INTERP/DECIM/POLY), ARITHMETIC/COMPLEX/VECTOR ops (CADD/CMUL/MAC/...), built and verified module-by-module against Python golden models

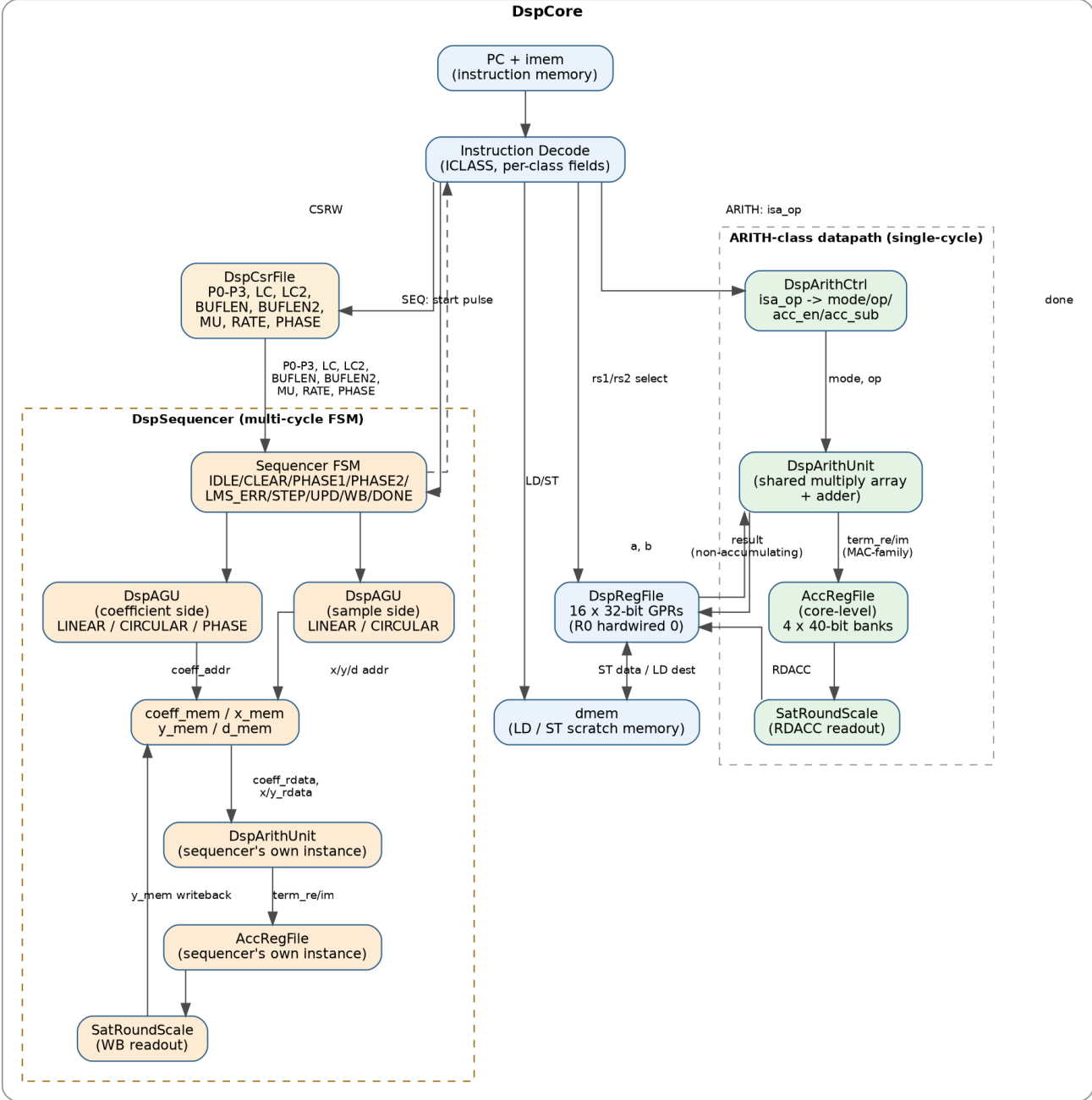
1. Overview - Two Nested FSMs

The processor runs one program-level instruction at a time in **DspCore**. Almost every instruction finishes in a single clock edge. The one exception is a **SEQ** instruction (a FIR/IIR/CONV/CORR/LMS/INTERP/DECIM/POLY call), which hands off to **DspSequencer** - a second, independent FSM that runs for anywhere from about 6 to 20+ cycles depending on tap count, while DspCore simply waits for it to finish. So there is an outer FSM (the program) and an inner FSM (one "bigger operation") that only runs while the outer FSM is parked on a SEQ instruction.

This document covers, in order: the top-level architecture (Section 2), DspCore's outer FSM and what each instruction class does (Section 3), DspSequencer's inner FSM shared by all 8 algorithms (Section 4), exactly how each of those 8 algorithms maps onto that one shared FSM (Section 5), and a full worked trace through a real assembled test program showing both FSMs interleave in practice (Section 6).

1.1 Top-Level Architecture

DspCore Top-Level Architecture



Two separate instances of the arithmetic cluster exist: one at the DspCore top level (used by single-cycle ARITH instructions and read back with RDACC) and one owned privately by DspSequencer (used only during a multi-cycle algorithm run). They do not share state - a FIR call's internal accumulator is invisible to RDACC.

2. Instruction Word Format

A first-draft 32-bit encoding, not fixed by any specific bit layout decided earlier beyond the mnemonic list and the dedicated P0-P3/LC/MU/RATE configuration registers, both of which this reuses directly. The top 3 bits (ICLASS) select one of 8 instruction classes; the remaining 29 bits are carved up differently per class.

ICLASS	Fields	What it does
000 ARITH	[28:25]=isa_op [24:21]=rd [20:17]=rs1 [16:13]=rs2 [12:11]=acc_sel	CADD/CSUB/VADD/VSUB/CONJ/MAG2/CMUL/VMUL/MAC/M SAC/CMAC/CMSAC/VMAC/VMSAC via DspArithCtrl's isa_op decode
001 SEQ	[28:26]=seq_mode [25:24]=acc_sel	FIR/IIR/CONV/CORR/DECIM/INTERP/POLY/LMS; operands come from the CSR file (P0-P3/LC/LC2/BUFLen/BUFLen2/MU/RATE/PHASE), configured beforehand with CSRW
010 LD	[28:25]=rd [24:21]=rs1(base) [20:9]=imm12	rd <= dmem[rs1+imm]
011 ST	[28:25]=rs_data [24:21]=rs1(base) [20:9]=imm12	dmem[rs1+imm] <= rs_data
100 LI	[28:25]=rd [24:9]=imm16	rd <= zero_extend16(imm16) - fills only the low half
101 PACK	[28:25]=rd [24:21]=rs1(hi) [20:17]=rs2(lo)	rd <= {rs1[15:0], rs2[15:0]} - builds a full complex/vector operand out of two scalars
110 CSRW	[28:25]=csr_sel [24:9]=imm16	CSR[csr_sel] <= imm16
111 SPECIAL	[28]=0: RDACC [27:24]=rd [23:22]=acc_sel [28]=1: HALT	RDACC: rd <= SAT/ROUND/SCALE(ACC[acc_sel]). HALT: core stops.

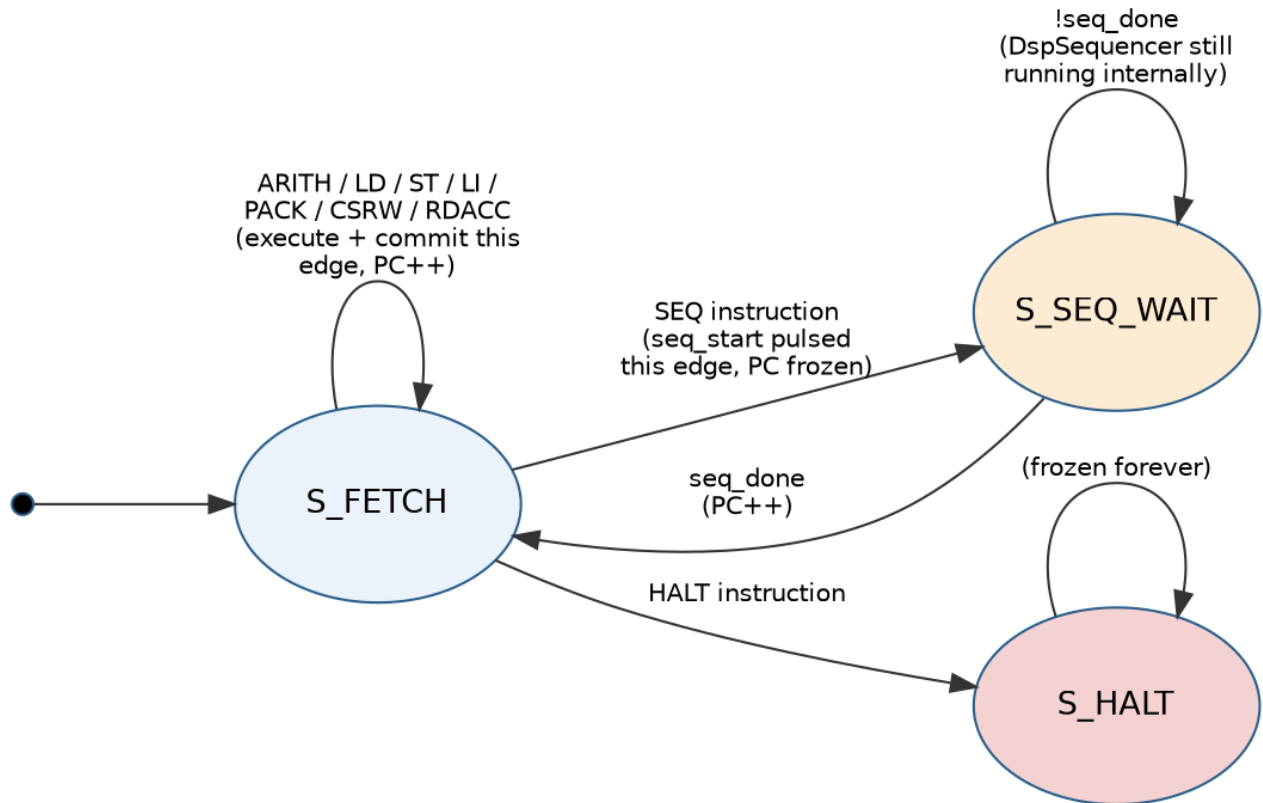
2.1 CSR Register Indices

DspCsrFile holds the 11 dedicated configuration registers a SEQ instruction reads from: index 0-3 = P0/P1/P2/P3, 4 = LC, 5 = LC2, 6 = BUFLen, 7 = BUFLen2, 8 = MU, 9 = RATE, 10 = PHASE. Software sets these with CSRW before issuing the SEQ instruction that consumes them.

3. DspCore - the Outer FSM

Three states: S_FETCH, S_SEQ_WAIT, S_HALT. Every cycle in S_FETCH, the current instruction word is decoded combinatorially by ICLASS, and one of three things happens.

DspCore Outer FSM (one instruction at a time)



3.1 Step-by-Step per Instruction Class

For every class except SEQ and HALT, the entire instruction executes combinatorially in the same cycle it is fetched, and commits at the same clock edge that also increments PC:

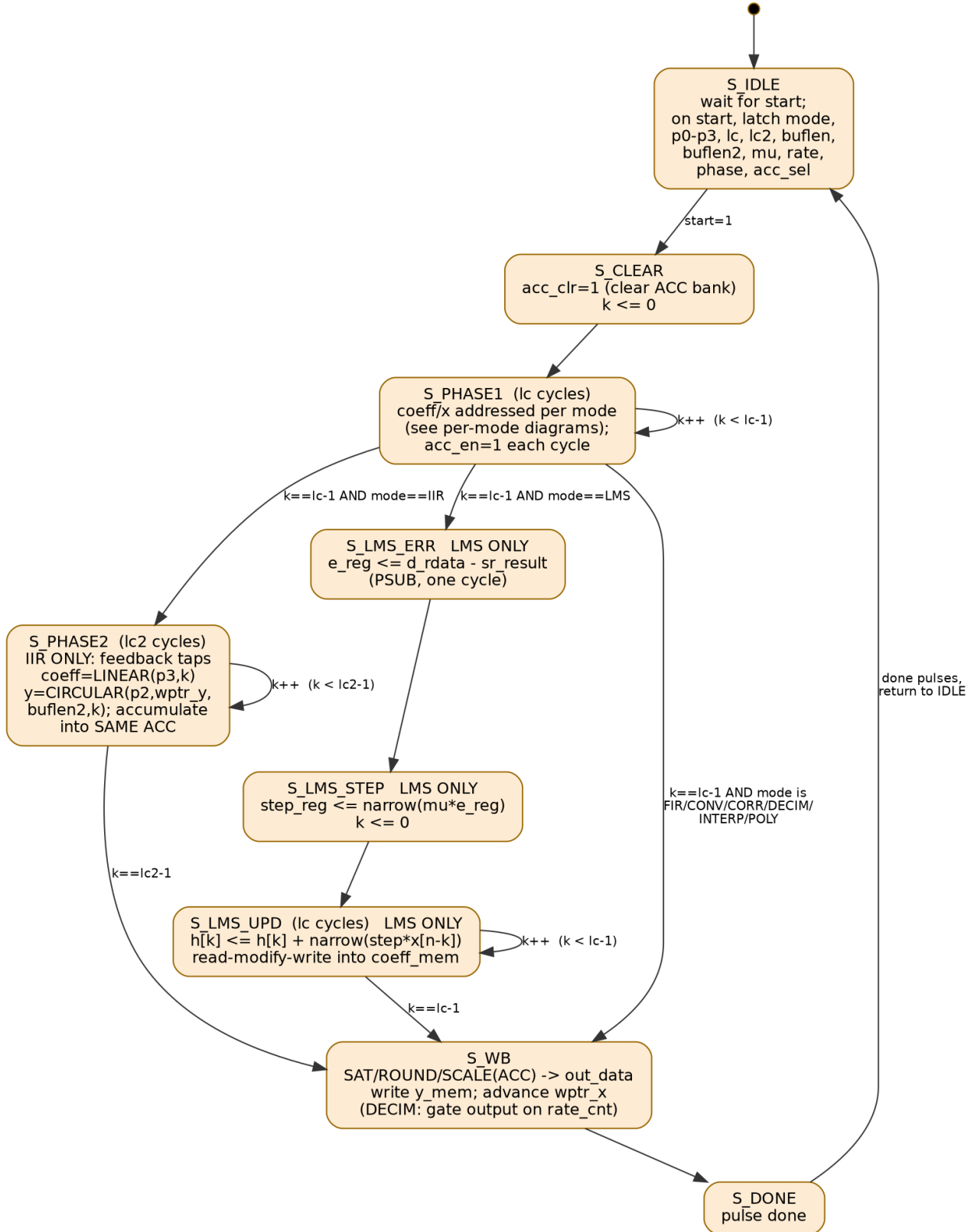
Class	Steps this cycle
ARITH	Read GPR[rs1]/GPR[rs2] → DspArithCtrl decodes isa_op into {mode,op,acc_en,acc_sub} → DspArithUnit computes combinatorially → if acc_en=0, write `result` into GPR[rd] this cycle; if acc_en=1 (MAC-family), instead pulse core_acc_en and accumulate term_re/term_im into the core-level ACC bank (no GPR write).
SEQ	Drive seq_start=1 combinatorially → DspSequencer (in its own S_IDLE) latches mode/p0-p3/lc/.../acc_sel and begins → DspCore moves to S_SEQ_WAIT with PC frozen → every cycle just poll seq_done → on done, PC++ and return to S_FETCH.
LD	dmem[rs1+imm12] read combinatorially → written into GPR[rd] this same cycle.
ST	GPR[rs2] (the data register) → written into dmem[rs1+imm12] at this clock edge.
LI	GPR[rd] <= {16'd0, imm16} - zero-extended into the low half only.

Class	Steps this cycle
PACK	GPR[rd] <= {GPR[rs1][15:0], GPR[rs2][15:0]} - the only way to build a full 32-bit packed complex/vector constant, since LI alone can't reach the high half.
CSRW	CSR[csr_sel] <= imm16, one of the 11 config registers.
RDACC	Core-level ACC[acc_sel] read combinatorially → run through a dedicated SatRoundScale (shift=0, round-nearest) → narrowed 16-bit result written into GPR[rd].
HALT	halted <= 1, move to S_HALT - a dead end, nothing changes from there.

4. DspSequencer - the Inner FSM (shared by all 8 modes)

Nine states carry every one of the 8 "bigger operations." What differs between modes is only which DspAGU addressing primitive feeds PHASE1/PHASE2 (see Section 5) - the loop control itself is identical hardware for every mode that shares a shape.

DspSequencer Inner FSM (the ONE shared sequencer for all 8 modes)



4.1 State-by-State

State	What happens
S_IDLE	Wait for start. The instant start=1, latch EVERYTHING the whole run will need - mode, p0-p3, lc, lc2, buflen, buflen2, mu, rate, phase, acc_sel - all in one edge. This is why the CSR/mode inputs only need to be valid for that one instant, not held for the whole run.
S_CLEAR	One cycle. acc_clr=1 zeros the selected AccRegFile bank; tap counter k resets to 0.
S_PHASE1	lc_r cycles, k=0..lc_r-1. Two DspAGU instances generate the coefficient address and the sample address; DspArithUnit multiplies them (always MODE_SCALAR/OP_MUL); acc_en=1 accumulates the raw product every cycle. On the last tap, branches by mode: IIR → S_PHASE2, LMS → S_LMS_ERR, everything else → S_WB.
S_PHASE2 (IIR only)	lc2_r cycles. Coefficient side LINEAR(p3,k) (the a[] feedback coefficients), sample side CIRCULAR(p2,wptr_y,buflen2,k) (y[n-1],y[n-2],...). Accumulates into the SAME ACC bank PHASE1 left running - feedforward and feedback land in one sum with no separate adder.
S_LMS_ERR (LMS only)	One cycle. d_rdata (desired signal, fixed address p2) minus sr_result (PHASE1's y[n], continuously available off the ACC) via DspArithUnit op=PSUB - the SAME unit PHASE1 just used for multiplying. Latched into e_reg.
S_LMS_STEP (LMS only)	One cycle. mu_r*e_reg, narrowed to Q1.15, latched into step_reg. k resets to 0.
S_LMS_UPD (LMS only)	lc_r cycles. Each cycle: read coeff_mem[p0+k] (old h[k]), multiply step_reg*x[n-k] through the same arithmetic unit, add to the old coefficient with one small DEDICATED 16-bit adder (cheap enough not to share), write back with coeff_we=1 at the same address - a genuine same-cycle read-modify-write.
S_WB	One cycle, five things at once: read ACC through SatRoundScale onto out_data; write it into y_mem at the next slot; advance wptr_x (INTERP/POLY: only on the LAST phase); advance wptr_y; for DECIM, gate whether this call's output is actually marked valid at all.
S_DONE	One cycle, pulses done, returns to S_IDLE.

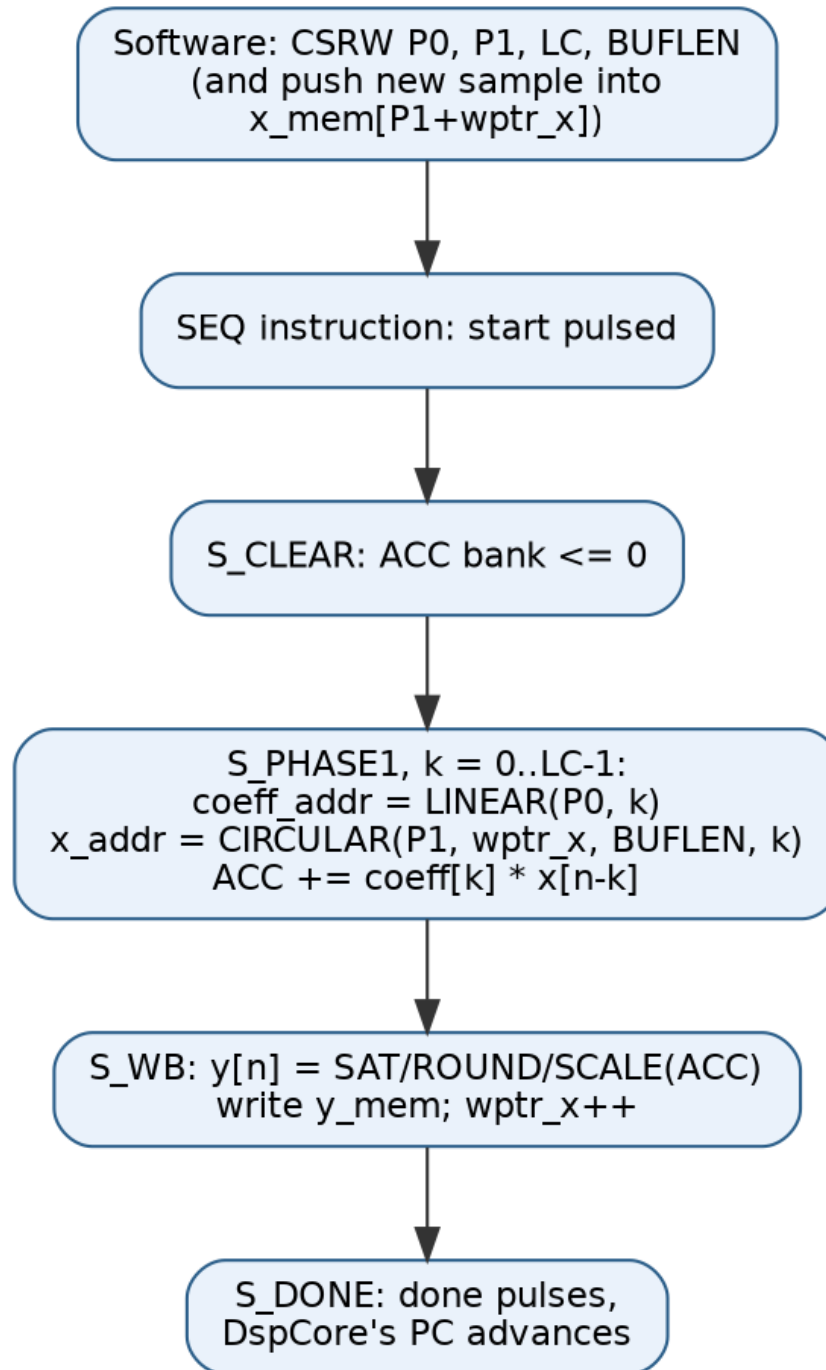
The one real bug caught while building this: x-history must NOT advance until the LAST phase of an INTERP/POLY group has used it - advancing on phase==0 (the first phase) would shift the window out from under the phases still to come. Fixed as **advance_x = (phase_r == rate_r-1)**.

5. How Each of the 8 Operations Maps onto the Shared FSM

Two genuinely distinct "shapes" cover all 8 mnemonics: Shape A (FIR, CONV, CORR, DECIM, INTERP, POLY) is just the PHASE1 MAC loop; Shape B (IIR, LMS) adds exactly one extra phase. Within Shape A, CONV/CORR turn out to need IDENTICAL wiring to each other (both are a dot product of two linearly-addressed buffers), and so do INTERP/POLY (both are phase-addressed coefficient banks with the sample window held stable across a group of calls) - so the 8 mnemonics reduce to 6 distinct wiring diagrams below.

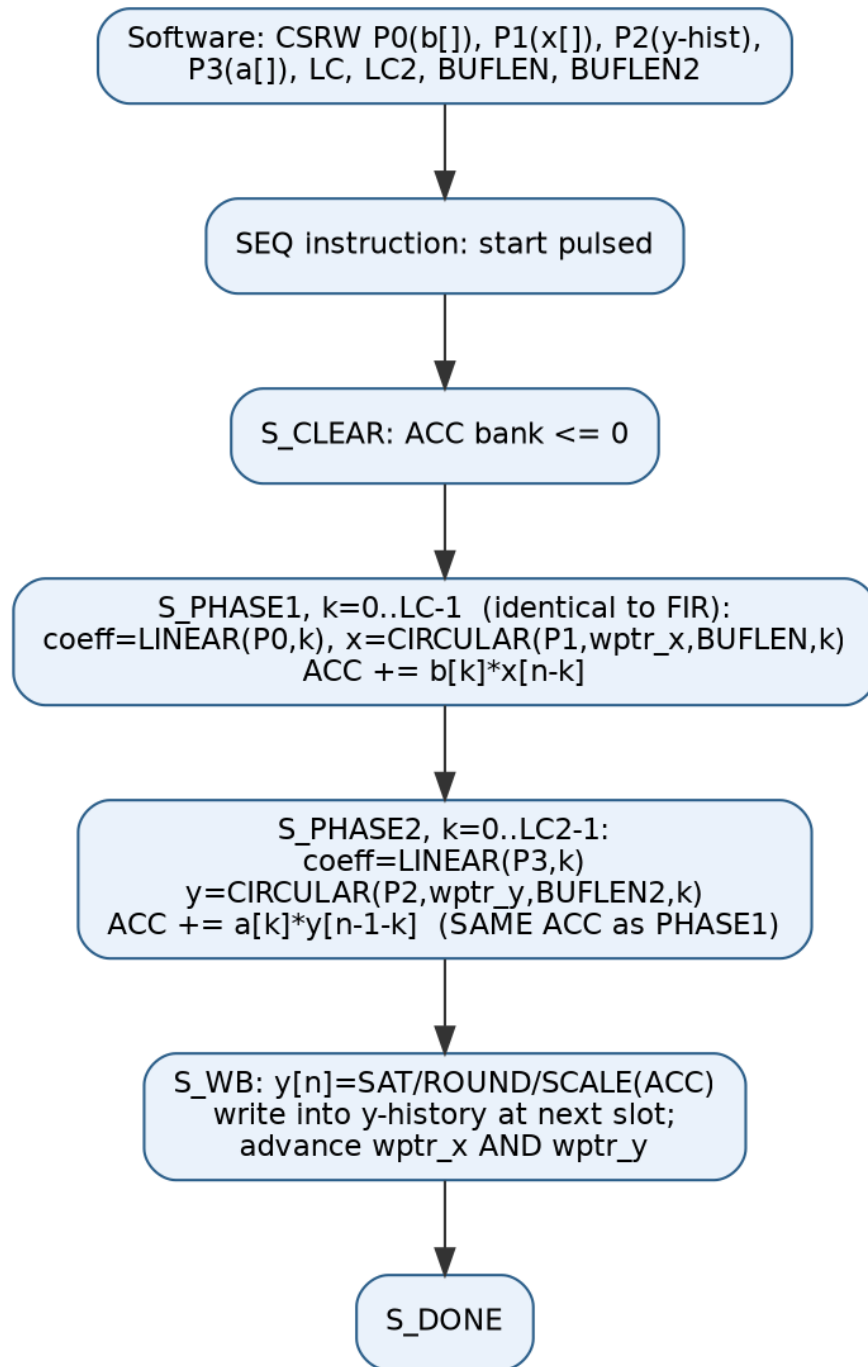
5.1 FIR (Shape A baseline)

FIR (Shape A baseline)



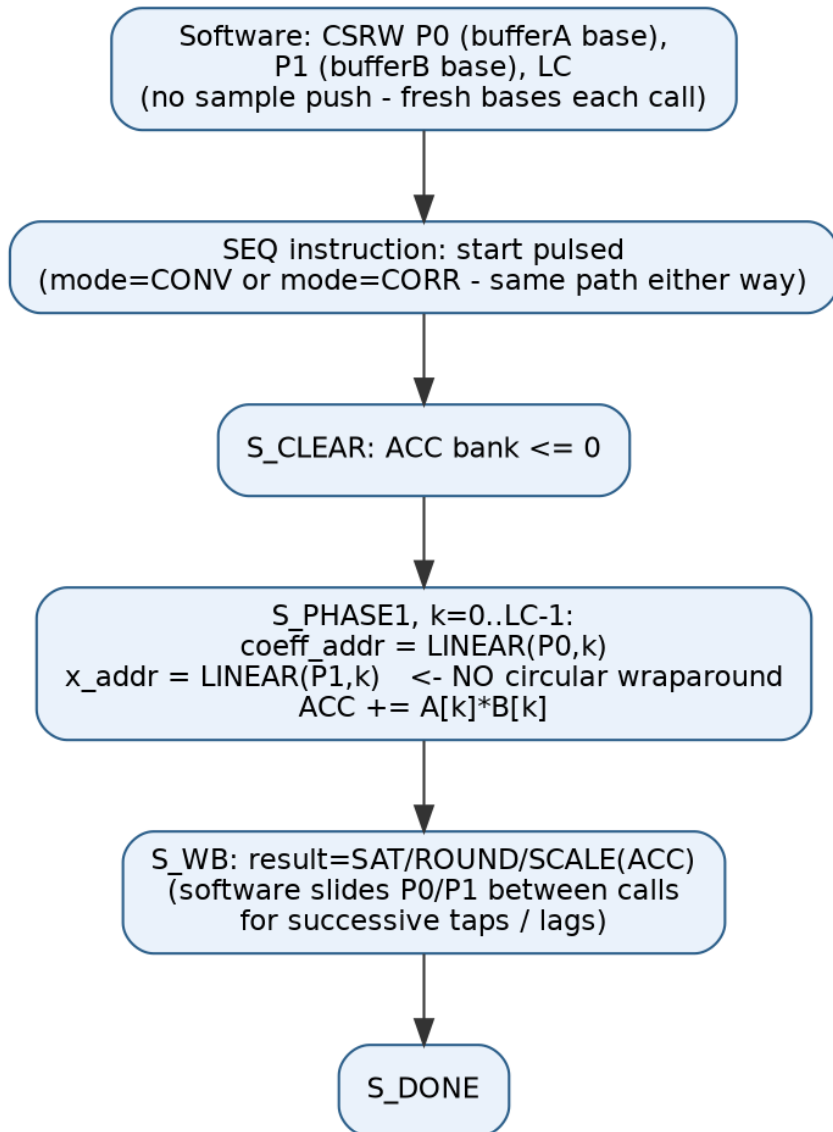
5.2 IIR (Shape B - feedback phase)

IIR (Shape B: feedback phase)



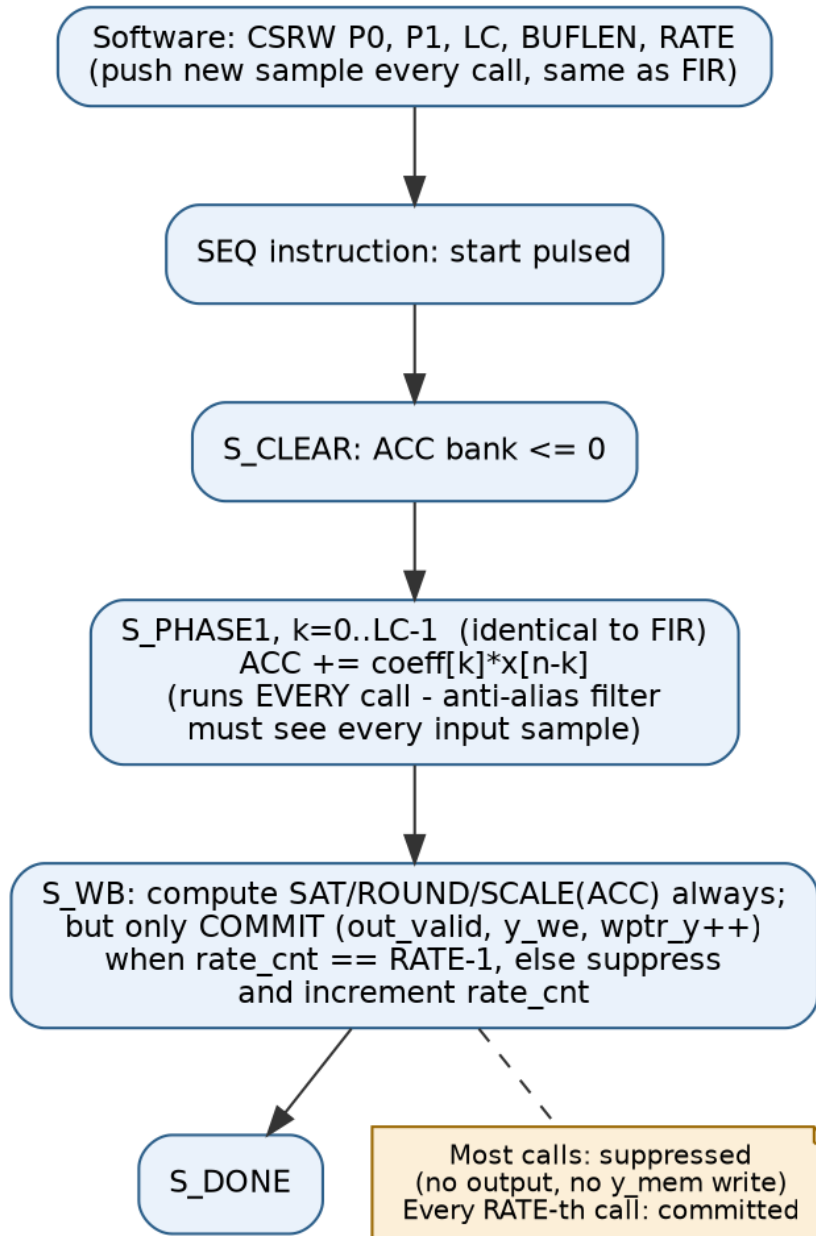
5.3 CONV and CORR (identical wiring)

CONV and CORR (identical hardware wiring)



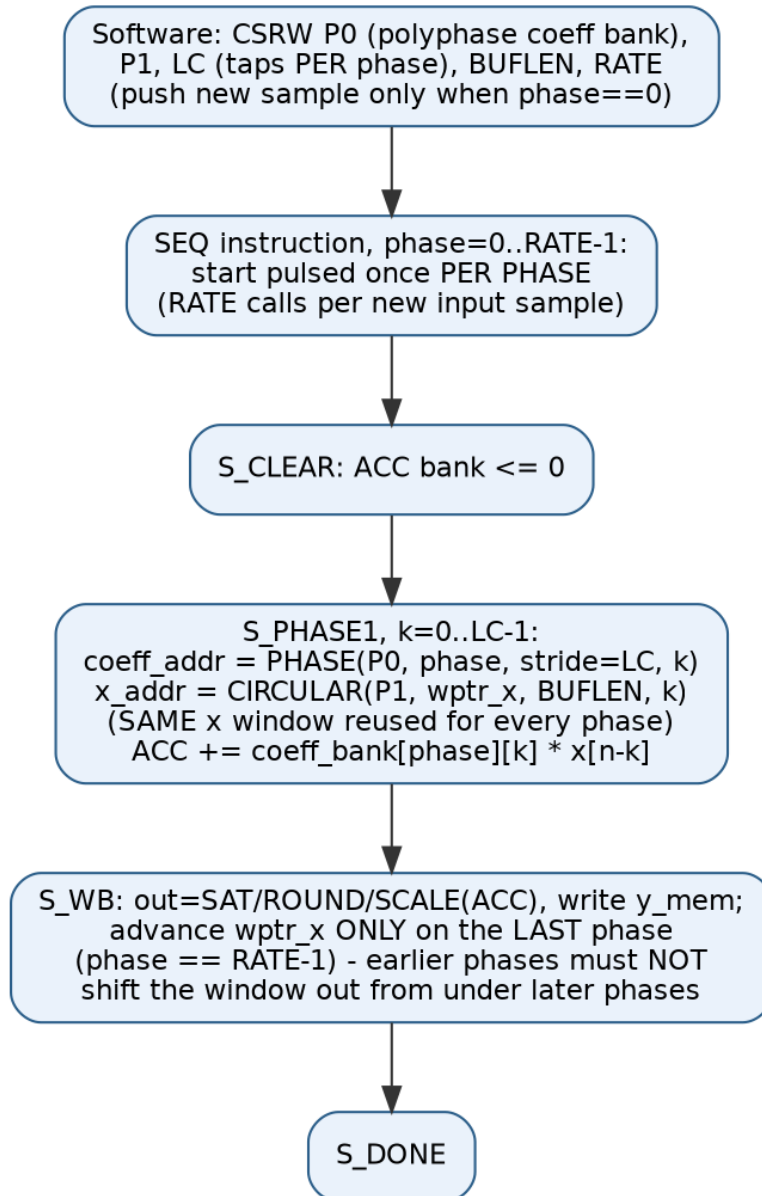
5.4 DECIM (Shape A + rate-gated writeback)

DECIM (Shape A + rate-gated writeback)



5.5 INTERP and POLY (identical wiring)

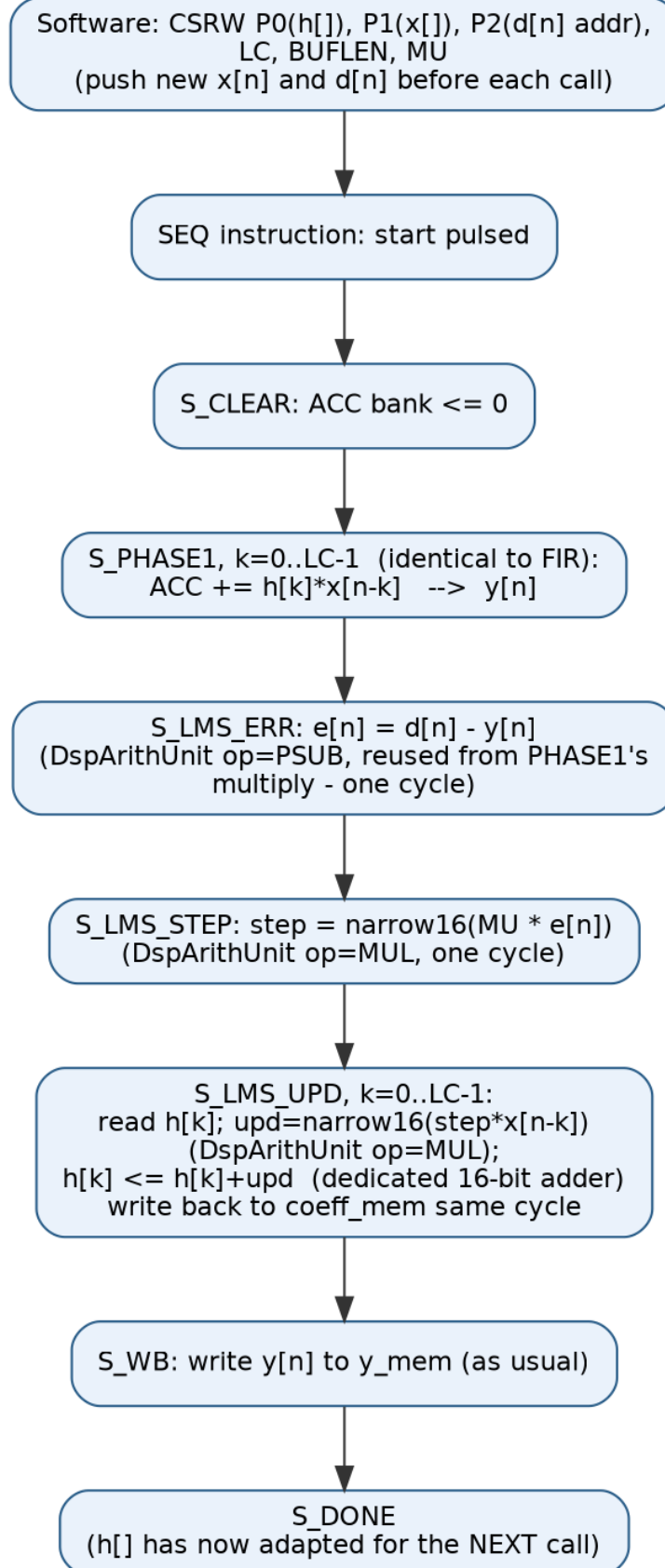
INTERP and POLY (identical hardware wiring)



5.6 LMS (Shape B - coefficient-update phase)

The most involved of the 8: on top of FIR's own PHASE1, LMS adds three more states (S_LMS_ERR, S_LMS_STEP, S_LMS_UPD) that compute the error, scale it by the step size, and read-modify-write every coefficient - all reusing the SAME DspArithUnit instance PHASE1 used, just fed a different op each state.

LMS (Shape B: coefficient-update phase)



6. Worked Trace: a Real Assembled Program

verify/gen_core_program.py hand-assembles an 18-instruction program exercising every instruction class in one run, and tb_dsp_core.v checks the final state against a Python golden model. Here is how the two FSMs actually interleave while running it.

Instr #	Instruction	DspCore state	What happens
1-6	CSRW x6	S_FETCH	Configure P0=P1=P2=0, LC=5, BUFLen=5, BUFLen2=8 - one cycle each, PC advancing every time.
7	SEQ(FIR, acc_sel=1)	S_SEQ_WAIT (PC frozen at 7)	DspSequencer runs S_IDLE→S_CLEAR→S_PHASE1 (5 cycles, k=0..4)→S_WB (writes y_mem[1]=0x02f6)→S_DONE - about 8 sequencer cycles elapse while DspCore just watches seq_done.
8-9	LI R1, LI R2	S_FETCH	One cycle each, load the CADD operands into the low halves of R1/R2.
10	CADD R3,R1,R2	S_FETCH	DspArithCtrl decodes acc_en=0 for CADD; DspArithUnit computes; R3 written this cycle.
11-12	ST R3→dmem[5], LD R4←dmem[5]	S_FETCH	One cycle each - proves the round-trip; R4 ends up equal to R3.
13-14	LI R6, LI R7	S_FETCH	Load the MAC operands.
15-16	MAC (acc_sel=2) x2	S_FETCH	Two SEPARATE instructions, each one cycle, both targeting the SAME core-level ACC bank 2 - the second MAC's accumulate lands on top of the first's, proving the accumulator genuinely persists across instructions (not just within one multi-cycle op).
17	RDACC R8, acc_sel=2	S_FETCH	Reads the accumulated sum through SatRoundScale into R8, one cycle.
18	HALT	S_FETCH → S_HALT	Core freezes.

Total: 18 instructions, but about 27 clock cycles - the one SEQ instruction burns roughly 8-9 cycles internally while DspCore's PC does not move. That matches exactly what tb_dsp_core.v reports (**halted after 27 cycles**), and every piece of final state - R3, R4, R8, dmem[5], and y_mem[1] (the actual FIR output) - matched the golden model bit-exactly on the first real run.

12 testbenches | 4 arithmetic-cluster + 1 AGU + 6 sequencer end-to-end (all 8 modes) + 1 full-core program | RESULT: ALL TESTS PASSED

Scoping Note

DspCore is single-cycle-per-instruction (multi-cycle only for SEQ, which genuinely needs it), not yet pipelined - deliberately, the same order the RISC-V half of this project went (single-cycle CPU first, pipelined with its own HazardDetectionUnit second). Pipelining a stream that mixes 1-cycle ARITH instructions with variable-latency SEQ instructions would need its own hazard/stall design - an analogue of HazardDetectionUnit.v for "sequencer busy" - which is a natural next milestone once this level is reviewed.